

四川麻将训练助手

Sichuan Mahjong Training & Real-time Assistant

课 程：	程序设计实习
组 名：	三人帮
组 长：	薛翔元
组 员：	张润泽 于天行
语 言：	C++ / Qt 6
赛道：	生活工具
日 期：	2026 年 6 月

一、程序功能介绍

1.1 项目背景

四川麻将是国内流行的地方麻将变种之一，其核心规则为"血战到底"——三家定缺、缺门才能和牌、一人和牌后其余玩家继续战斗。对于新手而言，判断听牌、选择舍牌、推测对手牌型都存在较高的学习门槛。本项目旨在为四川麻将玩家（尤其是初学者）提供一个集**练习、模拟、实时辅助**于一体的桌面工具。

1.2 核心功能

(1) 舍牌训练模式

系统随机生成 14 张手牌（清一色），玩家选择一张舍牌；程序自动计算该舍牌后的听牌种类与剩余张数，并与最优解对比，给出对错判断和详细分析。此模式帮助玩家建立舍牌直觉，理解牌效概念。

(2) AI 对局模拟

完整的四人血战到底对局：3 个 AI 对手，支持定缺、碰、杠（明杠/暗杠/加杠）、自摸/点和，一局多胜。玩家可随时"偷看"任意 AI 的手牌（作弊模式），观察 AI 在不同局势下的决策逻辑。支持自动托管（autopilot），一键全自动观战。

(3) 实时辅助模式

面向实战场景：用户手动输入当前手牌、定缺花色及场上已现弃牌，系统每轮给出**舍牌建议**（按向听数排序，标注孤立牌、边缘牌、缺门牌）、**对手危险度分析**（根据弃牌序列推断对手攻防风格与危险张）。支持**语音指令**快速录入（"摸 3 筒""打 7 万""下家碰 2 条"等），降低操作延迟。

(4) 对手分析工具

独立的分析面板：输入自己手牌与对手风格、定缺、弃牌、副露信息，输出每位对手的危险张排名和全局策略建议（防守/对攻）。

1.3 界面概览

主窗口包含两个标签页——"实战训练"（含实时辅助与 AI 对局两个子模式）和"模拟训练"（舍牌训练与四人全自动模拟）。所有牌面使用 SVG/PNG 渲染，支持悬停高亮与点击选牌。背景播放《赌神》主题音乐增强氛围。

二、项目各模块与类设计细节

2.1 整体架构

项目采用分层架构：纯算法层 → 游戏引擎层 → AI 决策层 → UI 展示层。各层职责清晰，下层不依赖上层。

2.2 数据结构层

Tile 结构体：表示单张麻将牌，包含花色（`TileSuit` 枚举：Dot/Bamboo/Character）和数值（1–9）。重载了 `==`、`<`，提供 `toString()` 方法输出“1筒”“2条”等中文表示。

PlayerState 结构体：每玩家的完整状态——手牌、副露（碰/杠的牌组）、弃牌池、定缺花色、是否定缺完成、是否已和牌。支持快照式状态查询。

GameContext 结构体：AI 决策所需的 game 快照——当前阶段、场上最后弃牌、弃牌玩家、各玩家存活/定缺状态等。

2.3 核心算法层 —— MahjongLogic

纯静态工具类，无 Qt 依赖，所有函数均可独立测试。

功能	核心方法	算法简述
牌墙生成	<code>generateWall()</code>	108 张牌（3 花色 × 9 数值 × 4 副本），Fisher–Yates 洗牌
和牌判定	<code>canWin()</code> <code>isWinningHand()</code>	30 元素计数数组（索引 0–9=筒，10–19=条，20–29=万）。递归：先试雀头（pair），再回溯消去顺子和刻子
听牌计算	<code>getWaits()</code>	遍历 27 种牌，逐一添加到手牌后调用 <code>canWin()</code>
舍牌建议	<code>getBestDiscardOptions()</code> <code>getDiscardSuggestions()</code>	逐张舍牌后计算听牌种类与剩余张数，按听牌总数降序排列
向听数	<code>calculateShanten()</code> <code>computeShanten13()</code>	公式：8 – 2×面子 – 有效搭子 – 雀头加成。逐花色的递归搜索最优面子和搭子排列（ <code>searchSuit()</code> ）
番型计算	<code>calculateFanTypes()</code>	基础 1 番，清一色×4，七对×4

危险度分析	<code>analyzeDangerTiles()</code>	Level-1 模型：生张基础危险度 × 持牌封锁因子 × 邻张关联因子 × 缺门过滤
训练牌生成	<code>generateTrainingHand()</code>	保证生成的 14 张手中至少有一张舍牌能进入听牌

2.4 游戏引擎层 —— `GameEngine`

`QObject` 子类，状态机驱动。管理发牌→定缺→对局→结算的完整流程。

阶段流转（Phase 枚举）：`Idle` → `Dealing` → `Dingque` → `Playing` → `Finished`

关键信号：`gameStarted`、`turnChanged(int player)`、`playerDiscarded(int, Tile)`、`playerWon(int, bool selfDrawn, int fan)`、`gameOver`。UI 层通过信号-槽机制响应游戏事件，实现解耦。

动作校验：`canPong()`、`canExposedKong()`、`canSelfKong()`、`canWinOnDiscard()` 在每次轮转后自动计算，仅展示合法选项。

血战到底：`checkGameEnd()` 在玩家和牌后检查是否仅剩一人存活或牌墙耗尽，否则继续对局。

2.5 AI 决策层 —— `AIPlayer`

基于规则的 AI，模拟中等水平玩家的决策逻辑：

决策	策略
定缺	选手中张数最少的花色，减少弃牌成本
舍牌	优先弃缺门牌 → 孤立安全牌 → 其他孤立牌 → 听牌数最低的非孤立牌
碰牌	非缺门花色 && 碰后手牌 ≥ 4 张 → 碰
明杠	非缺门花色 && 当前未听牌 → 杠（保留牌型结构）
暗杠/加杠	当前未听牌 → 杠
和牌	机会出现即和

2.6 UI 展示层

类名	职责	代码行数
----	----	------

MainWindow	主窗口骨架, QTabWidget 容器 (实战训练 / 模拟训练)	~80
BattleTrainingWidget	模式选择页 (实时辅助 / AI 对局), QStackedWidget 切换	~60
CombatArenaWidget	核心游戏界面: 对局面板、对手展示、手牌交互、建议面板、抓包/语音输入、背景音乐	~2557
TrainingWidget	舍牌训练: 出题、校验、最佳解反馈	~300
SimulationWidget	四人全自动模拟: 完整牌桌布局、自动托管开关	~600
OpponentAnalysis	独立对手分析面板: 手牌/风格/定缺/弃牌输入 → 危险张排名	~300
RealTimeAssistant	实时辅助原型: 训练手牌 + 舍牌建议展示	~150

关键设计点:

- **信号-槽解耦:** CombatArenaWidget 连接 GameEngine 的信号, 状态变更自动驱动 UI 刷新, 无需轮询。
- **牌面渲染:** tileToHtml() 将 SVG/PNG 牌面以 base64 嵌入 HTML, 通过 QLabel::setText() 渲染为 RichText, 实现灵活的牌面布局。
- **语音指令解析:** onVoiceCommandSubmit() 通过正则匹配"摸 3 筒""打 7 万""下家碰 2 条"等模式, 映射到 GameEngine 对应操作。

三、小组成员分工情况

成员	角色	负责模块	主要工作内容
薛翔元 (组长)	架构 & 算法	mahjonglogic gamengine aiplayer	- 核心麻将算法：和牌判定、听牌计算、向听数、番型计算、危险度分析 - 游戏状态机设计：阶段流转、动作校验、血战到底规则 - AI 玩家：定缺/舍牌/碰杠/和牌决策策略 - 项目架构设计、GitHub 仓库管理、代码审查
张润泽	训练 & 分析	trainingwidget simulationwidget opponentanalysis realtimeassistant	- 舍牌训练模式：出题逻辑、对错判定、最优解展示 - 四人全自动模拟：完整牌桌 UI、自动托管 - 对手分析工具：独立分析面板、风格推断、危险张排名展示 - 实时辅助原型开发 - 牌面素材整理（27 张 SVG 牌面）
于天行	主界面 & 游戏	mainwindow battletrainingwidget combatarenawidget	- 主窗口框架与标签页容器 - 实战训练界面：对局面板、手牌交互（点击选牌/悬停高亮）、对手展示区 - 实时辅助模式：手动抓包对话框、轮次更新、语音指令解析 - 舍牌建议面板、对手分析面板渲染 - 背景音乐集成（dushen.mp3 循环播放） - 牌面 SVG/PNG 渲染与 base64 嵌入

注：以上分工为模块主导分工。实际开发中，全体组员均参与了跨模块联调、Bug 修复和代码评审。

四、项目总结与反思

4.1 技术收获

1. **面向对象设计实践**：从数据结构层到 UI 层的四层架构使代码职责清晰，每个类的修改不影响其他层。例如 `MahjongLogic` 的向听数算法优化不需要改动任何 UI 代码。
2. **Qt 信号-槽机制**：`GameEngine` 通过信号驱动 UI 刷新，避免了传统轮询模式的性能开销和耦合问题。理解并实践了"模型发出信号、视图响应信号"的设计范式。
3. **麻将规则的形式化建模**：将四川麻将的"定缺""血战到底""缺门才能和牌"等口语化规则转化为精确的程序逻辑，需要细致的状态管理和边界条件处理（如多家同时可碰/杠/和的优先级）。
4. **递归搜索与剪枝**：向听数计算中的逐花色递归搜索是项目中算法复杂度最高的部分。通过合理的剪枝（无效搭子提前终止）将搜索空间控制在可接受范围。

4.2 项目不足

1. **危险度模型较简单**：当前的 Level-1 模型仅考虑生张、封锁、邻张关联和缺门过滤，未引入更复杂的读牌逻辑（如根据对手碰牌推断手牌结构）。后续可用贝叶斯推断或蒙特卡洛模拟提升准确度。
2. **番型覆盖不完整**：目前仅支持清一色和七对两种番型，对对胡、带幺九、杠上开花等尚未实现。这是四川麻将计分体系的重要组成。
3. **AI 缺乏适应性**：AI 始终使用固定策略，不会根据对手风格（激进/保守）调整行为，长期对局中可预测性较强。
4. **牌面 UI 自适应不足**：当前界面以固定分辨率设计，在小屏幕或高 DPI 环境下可能存在布局问题。

4.3 如果重来

- 在项目初期建立更完善的单元测试（特别是 `MahjongLogic` 的和牌与向听数计算），避免后期回归测试依赖人工逐个场景验证。
- 更早引入 Git 分支工作流（feature branch + PR review），而不是直接在 main 分支开发。
- 预留更充分的时间做路演准备——演示脚本设计和录屏质量对最终展示效果影响很大。

4.4 致谢

感谢程设课程助教在项目立项和中期检查中提出的建议，帮助我们在功能范围和实现深度之间找到了合理的平衡。

— 报告完 —